Apache Paimon C++ ：从阿里数据湖走向开源社区

刘欣瑀 阿里技术 2026年9月9日 03:18 浙江 2人



这是2026年的第 59 篇文章

（ 本文阅读时间：约 20 分钟 ）

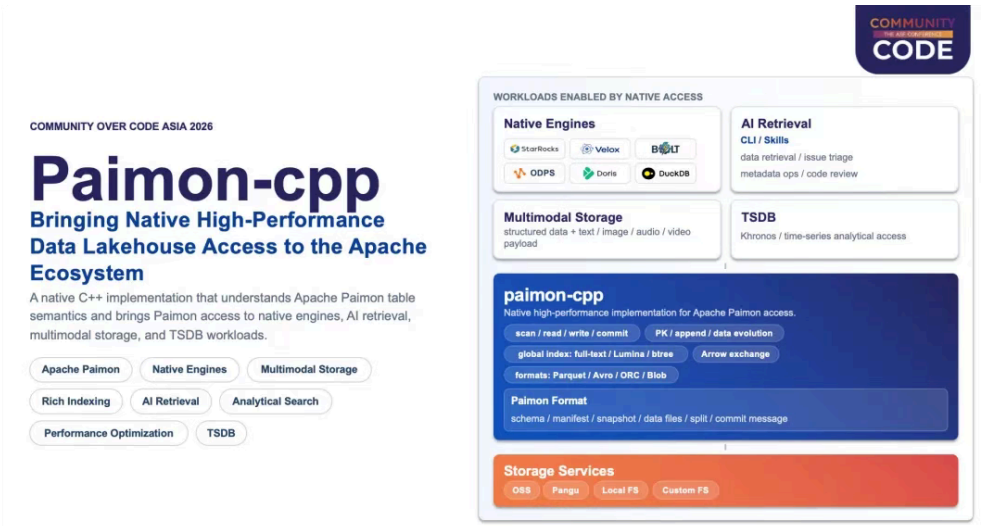
前言

Apache Paimon C++ v0.3.0 最近发布了。这是 Paimon C++ 在 Apache 基金会下的第一个正式版本，也意味着一个从阿里内部数据湖需求里长出来的项目，开始以社区化的方式面向更多引擎和开发者。

把时钟拨回 2024 年 1 月。阿里巴巴集团决定统一采用 Apache Paimon 作为数据湖表格式。但集团内部跑着的 ODPS、StarRocks、Native Flink、Native Spark 等，大多是 C++ Native 引擎。它们需要的不只是一个能够打开 Parquet 或 ORC 文件的 Reader，是一套不依赖 JVM、能够完整理解 Paimon 表语义，并充分发挥 Native 引擎性能的读写库。

Paimon C++ 由此出现。项目由阿里巴巴智能引擎承担，团队长期为淘宝、高德、菜鸟、钉钉、1688、闲鱼等内部搜推广业务提供高效稳定的工程能力。项目没有简单移植 Java 实现，系统性调研了不同引擎的执行模型和真实 Workload，设计了**一套以 Arrow 批式数据交换为基础、支持深度插件化定制的 Native Paimon C++ SDK。**

经过近两年生产验证，Paimon C++ 已在阿里数据湖上稳定支撑数 EB 数据。2025 年 12 月在 GitHub Alibaba 开源后，项目很快吸引到蚂蚁、字节、云器等团队参与共建。如今进入 Apache 社区，v0.3.0 成为新的起点。



01
一套已经服务多类 Native Workload 的 Paimon SDK

在进入 Apache 社区之前，Paimon C++ 已经在不同类型的引擎、产品和业务中形成接入与生产实践：

- **大规模生产使用**：Native Spark（Gluten + Velox）、StarRocks、Native Flink 已在生产环境大规模使用 Paimon C++ 读写能力；
- **开源社区接入**：已形成 DuckDB 社区扩展、字节跳动 Bolt、Apache Doris 等社区生态接入；
- **云上产品接入**：StarRocks、Milvus、DuckDB、Spark 和 Hologres 等云上产品正在接入或使用 Paimon C++；
- **内部业务使用**：蚂蚁集团（Spark、OceanBase、Explorer）、字节跳动 TokaDB、云器等内部业务已经开展使用和验证；
- **即将接入**：下一步将启动 ClickHouse Native 接入，进一步覆盖高性能 OLAP 查询场景。

这些接入场景的执行框架、数据格式、文件系统、内存管理和查询模式并不相同。Paimon C++ 的目标是让它们保留自己的性能优势，同时共享同一套 Paimon 表语义。

02
从阿里巴巴真实需求中诞生

集团统一采用 Paimon 后，Native 引擎面临的第一个问题是：如何正确、高效地访问 Paimon 表？

直接读取对象存储中的 Parquet 或 ORC 文件并不够。Paimon 表中的有效数据并非「目录下全部文件」的简单集合，是由 Schema、Snapshot、Manifest、Partition、Bucket、索引以及表的更新与合并规则共同确定。引擎还需要正确处理：

- 当前 Snapshot 对应哪些数据文件；

- Append 表和主键表各自的读取、更新与合并语义；
- Merge-on-Read、Deletion Vector 和 Compaction 如何生成最终结果；
- 统计信息、文件索引和全局索引如何参与数据裁剪；
- 写入结果如何与 Paimon 的表版本及格式协议保持兼容。

如果每个引擎分别实现一套 Paimon Reader/Writer，不仅建设成本高，格式兼容性和长期维护也会成为问题；如果所有数据都绕回 JVM 处理，又很难充分发挥 Native 引擎已有的向量化执行和 I/O 优化能力。

因此，Paimon C++ 以“完整理解 Paimon 表语义”为目标：计算引擎继续负责分布式调度、算子执行和数据交换，Paimon C++ 则负责 Scan、Read、Write、Commit、Compaction 以及相关的格式和表语义。

03
C++ 引擎真正需要什么样的 Paimon SDK?

Paimon C++ 针对 Native 引擎的执行方式进行了专门设计，而非只是将 Java 实现迁移到 C++。经过对不同引擎使用场景的调研，一套适合 C++ 生态的湖表 SDK，需要同时满足以下四类诉求。

直接适配向量化执行

分析型引擎普遍以列式 Batch 为单位完成计算。Paimon C++ 采用 Arrow C Data Interface 作为主要数据交换边界，读取结果以 Arrow Batch 返回，写入侧也直接接收 Arrow Batch。

这种设计减少了逐行对象转换和不必要的数据复制，可以更自然地衔接向量化算子、批量调度和并行执行。同时，Arrow C Data Interface 避免将 Paimon C++ 绑定到某个特定 Arrow C++ SDK 版本，降低引擎已有 Arrow 依赖之间的 ABI 冲突和集成成本。

保留引擎已有的性能能力

不同引擎通常已经在文件读取、缓存、内存管理和任务调度上投入多年优化；接入新的湖表 SDK，不应该意味着放弃这些积累。

Paimon C++ 因此设计了面向 Native 引擎的插件化扩展机制，允许引擎接入自定义的：

- **Format Reader/Writer**：复用已有的 ORC、Parquet 或其他格式实现；
- **FileSystem**：复用已有的对象存储访问、缓存和限流能力；
- **MemoryPool**：接入引擎自己的内存统计、限额和回收体系；
- **Executor**：复用引擎内部的线程池与任务调度能力。
- **Indexer**：支持引擎按需定制索引实现。

例如，StarRocks 在接入过程中复用了内部带 Block Cache 的文件系统，使原有缓存与 I/O 优化能够继续作用于 Paimon 读写链路，避免因为接入 SDK 而丢失已有性能能力。

字节跳动 Bolt 则通过实现 Parquet 插件复用内部精心优化的 Parquet 读写能力，使其在接入 Paimon 表语义后，仍然能够延续原有的格式读取优势。

很多引擎还会注入自己的 MemoryPool，对 Paimon 读写链路的当前内存、峰值内存和分配行为进行统一观测，并与引擎内部的资源管理和调度策略协同。

同时提供高效的默认实现

充分开放定制能力，并不意味着所有引擎都必须自行实现底层组件。Paimon C++ 同时提供一套高效的默认实现，包括：

- ORC、Parquet、Avro、Blob 等文件格式插件；
- Local、Jindo、S3 等文件系统实现与统一文件系统抽象；
- 默认 MemoryPool 和 Executor；
- Manifest Cache、Parquet Metadata Cache 等元数据缓存；
- 文件索引，以及面向标量、全文和向量检索的全局索引。

引擎可以开箱即用，也可以只替换自己最关注的组件。Paimon C++ 希望在通用性与专属性能之间提供足够灵活的选择。

与 Apache Paimon 保持格式和协议兼容

Paimon C++ 原生实现 Paimon 表语义，同时保持与 Apache Paimon Java 格式及通信协议兼容，包括 Manifest、Data Split、Commit Message 等关键数据结构。

不同语言和引擎因此可以共享同一张 Paimon 表，而不需要分别维护相互割裂的格式分支。

04
Paimon C++ 的五个设计重点

Paimon C++ 目前的核心能力可以归纳为五点。

将完整 Paimon 表语义带入 C++ 生态

Paimon C++ 是一套覆盖 Scan、Read、Write、Commit、Compaction 等操作的表级 SDK，支持 Append 表和主键表，以及 Merge-on-Read、Deletion Vector 等关键语义。

设计基于 Arrow 的 Native 批式数据通路

通过 Arrow C Data Interface 在引擎与 SDK 之间交换列式 Batch，让 Paimon 数据可以直接进入 Native 引擎的向量化执行链路，减少逐行转换、数据复制和跨语言交互。

建立可深度定制的插件化架构

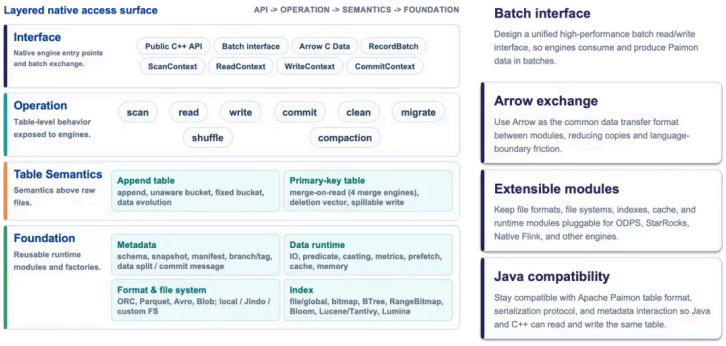
FileFormat、FileSystem、MemoryPool 和 Executor 均可由引擎定制。引擎可以复用经过生产验证的基础设施和优化经验，同时由 Paimon C++ 统一维护表格式、协议、索引和一致性语义。

围绕 Native 读写链路进行系统优化

Paimon C++ 围绕 Native 读写链路进行了多项系统优化，本文后续会介绍其中几项：读取侧的文件预取、Pipeline、多线程解码、异步 Row-to-Batch 和查询短路，以及主键写入侧通过分层 Spill Merge 减少临时文件重复读取和归并读放大。

从真实 Workload 出发扩展 Paimon 的能力边界

除了传统分析，Paimon C++ 还支持 Data Evolution、多模态数据和 Global Index，探索面向时序、动态 JSON、Free-schema 和实时数据等 Workload 的存储与检索模式，并推动成熟能力进入 Apache Paimon 标准演进。



05
完整读写链路如何工作

Scan 与 Read：让计划和执行自然分离

Paimon C++ 将读取拆分为 Scan 和 Read 两个阶段：

- **Scan**: 读取 Snapshot 和 Manifest，根据 Partition、Bucket、统计信息和索引裁剪无关文件，并生成可序列化的 Split；
- **Read**: 在不同 Worker 上并行读取 Split，执行列裁剪、嵌套字段裁剪、谓词下推、Page Filtering、预取和缓存，最终返回 Arrow Batch。

Coordinator 可以集中生成查询计划，Reader Task 则在不同节点并行执行。Split 可以跨进程和语言传递，数据通过 Arrow 批式进入后续计算链路。

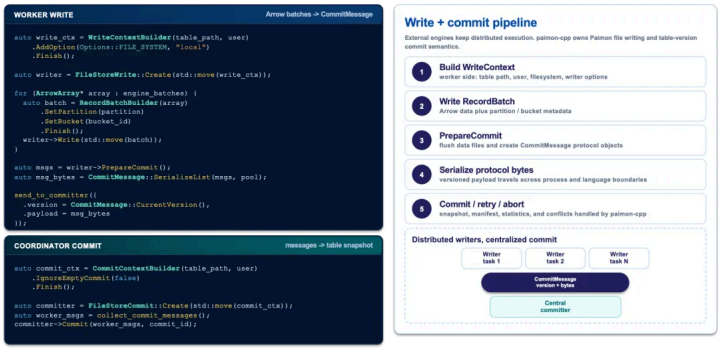


Write 与 Commit：保留引擎的分布式执行模型

写入侧同样采用适合分布式引擎的两阶段模型：

- **Write**：Worker 接收 Arrow Batch，完成数据组织与文件写入，并生成 Commit Message；
- **Commit**：Coordinator 收集各 Worker 的 Commit Message，由 Paimon C++ 处理 Manifest、统计信息、冲突检查和 Snapshot 提交。

引擎仍然负责调度和数据交换，Paimon C++ 则保证表版本、文件格式和提交语义的正确性。



目前，Paimon C++ 已支持 Append 表和主键表的读写，包括 Merge-on-Read、Deletion Vector 和 Compaction 等能力。

06
从传统分析走向 AI 与多模态检索

传统分析查询通常采用“列裁剪 + 谓词下推 + 批量扫描”的访问方式。AI Agent 面对的数据和查询模式正在发生变化：一次检索可能同时包含向量相似度、全文关键词、权限与时间等标量过滤，并最终返回文本、结构化字段、图片、音频或视频。

在 Data Evolution 表模式下，Paimon 为每行数据分配表级全局 Row ID，并通过 Row Tracking 保持稳定的行级定位能力。在此基础上，同一张 Paimon 表可以统一管理：

- 结构化数据；
- 文本与文档；
- Embedding 向量；
- 图片、音频、视频等 Binary / Blob 多模态数据。

查询模式也由传统扫描扩展为“全局索引召回 + Row ID 精准点查”：

1. Global Index 根据标量、文本或向量条件召回候选 Row ID 和可选 Score；
2. Paimon C++ 结合 Snapshot、Manifest 和文件范围生成 IndexedSplit；
3. Reader 根据 Row ID 精确读取所需字段或 Blob 数据。

Paimon C++ 当前提供：

- 基于 BTree 的标量查找与范围过滤；
- 基于 Lucene 和 Tantivy 的全文检索；
- 基于 Lumina 的向量相似度检索；
- 参与 Scan 和 Read 裁剪的各类文件索引。



这种模式让“一份数据入湖，全部数据可检索”成为可能。结构化数据、文本、向量和多模态 Payload 不再必须复制到多套专用系统中，可以减少数据复制带来的一致性、成本和运维问题。

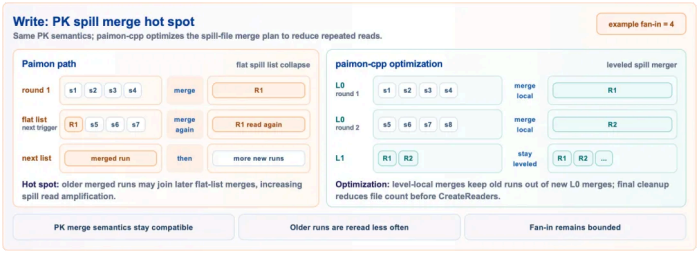
07
Native 读写链路的性能优化

Paimon C++ 的性能价值不只是去掉 JVM 和跨语言转换。项目还围绕 I/O、解码、数据复制、主键合并和框架数据转换进行了专门设计。

写入：降低 PK Spill Merge 的读写放大

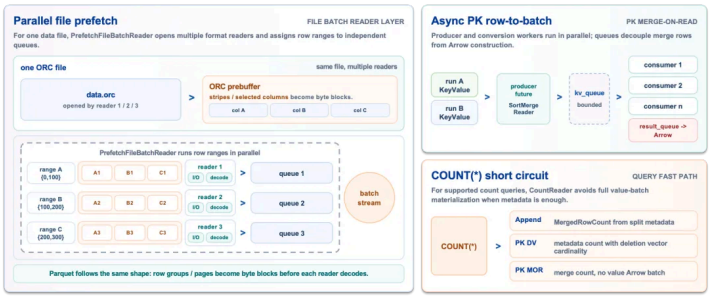
主键表写入需要对记录进行排序和归并。当内存 Buffer 超限时，中间结果需要 Spill 到临时文件。

传统平铺式 Spill List 在后续归并中，可能反复读取已经参与过归并的旧 Run，产生额外读放大。Paimon C++ 使用分层 Spill Merge，只归并达到 Fan-in 阈值的当前层级，减少旧 Run 的重复读取，并控制单次归并规模。这一优化不改变主键表语义，但能减少临时文件 I/O，并让归并过程在数据规模增大时保持更加稳定。



读取链路的主要优化

1. **并行文件预取**：同一 ORC 或 Parquet 文件可按 Row Range 分配给多个 Reader，并行完成 I/O、解码和结果输出；每个 Reader 线程内部还通过 Pipeline 重叠数据读取与解码过程，进一步降低查询延迟。
2. **异步 PK Row-to-Batch**：Sort Merge 生产端与 Arrow Batch 转换消费端异步，通过有界队列解耦。同时，消费端支持多线程进行 KeyValue 到 Arrow Batch 的转换。
3. **COUNT(*) Short Circuit**：对于 Append、PK + DV 和 PK MOR 场景，直接基于元数据或合并结果完成计数，避免构造完整的 Value Batch，以及由此产生的不必要的框架间数据转换开销。



此外，Scan 阶段会利用 Snapshot、Manifest、Partition、Bucket、统计信息和索引裁剪文件；Read 阶段继续进行列裁剪、嵌套字段裁剪、Predicate Pushdown、Page Filtering、Prefetch 和 Cache，从源头减少真正需要读取的数据量。

08
端到端性能验证

以下数据来自 Alibaba 集团生产环境，对比了相关引擎切换至 Paimon C++ 前后的端到端性能，反映了引擎完整查询链路的实际变化。实际效果会受到数据分布、文件数量、查询类型、硬件和引擎配置等因素影响。

Native Spark：Gluten + Velox

测试对象为 PK Merge-on-Read 表，数据规模 119.32 GB，约 70 个文件存在主键重叠。

场景	Paimon C++	切换前基线	提升
PK MOR 读取，8 轮	21～48 分钟	41～126 分钟	1.5～3.7 倍

StarRocks Native Reader

测试环境为单 BE、8 Core / 32 GB，对比 StarRocks 引擎切换 Paimon C++ 前后的端到端查询性能。

场景	Paimon C++	切换前基线	提升
Narrow Count	150 ms	220 ms	1.5 倍
PK Count，1000 列	403 ms	1251 ms	3.1 倍

Narrow Aggregate	202 ms	451 ms	2.2 倍
PK Point / Filter	231～274 ms	376～403 ms	1.4～1.7 倍
Full-width Aggregate	18.5 秒	62.0 秒	3.4 倍

这些结果说明，Paimon C++ 的价值不仅在于消除 JVM 边界，还来自对 Paimon 表语义的完整 Native 实现，以及围绕文件读取、主键合并和 Arrow 批式转换所做的系统优化。随着数据规模扩大，这些优化能够更充分地发挥作用，Paimon C++ 的性能收益也会更加明显。

09
从 Alibaba 到 Apache：持续演进与社区共建

从阿里内部数据湖业务到 Apache 社区，Paimon C++ 的核心目标没有变：让 Native 引擎在不丢掉自身性能优势的前提下，完整、稳定地接入 Paimon 生态；变化的只是参与的人越来越多，场景越来越广。

未来，社区会同步推进两条线：一条线紧跟 Apache Paimon 主线的格式和功能演进，完善 Native Scan、Read、Write、Commit、REST Catalog、Changelog 等能力；另一条线从真实需求中持续探索 AI、多模态、时序、动态 JSON、Free-schema 和实时数据等场景，把成熟经验沉淀为社区标准。

也欢迎来自查询引擎、数据湖、AI Infra、可观测和存储领域的开发者关注、试用、共建，一起完善 Apache Paimon 的开放生态！

相关链接

Apache Paimon C++ 官方仓库：https://github.com/apache/paimon-cpp

GitHub Release Note：https://github.com/apache/paimon-cpp/releases/tag/v0.3.0

使用文档：https://paimon.apache.org/docs/cpp/

Apache Paimon 官方网站：https://paimon.apache.org

Apache Paimon GitHub：https://github.com/apache/paimon

Apache Paimon C++ 交流群

欢迎加入 Apache Paimon C++ 钉钉交流群，与社区开发者一起交流使用经验、讨论技术问题，并参与社区共建。



使用钉钉扫描二维码，即可加入群聊



欢迎留言一起参与讨论~

阅读 1429

留言

写留言

还没有留言，来写首评